

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет систем управления и робототехники

Практическая работа $N_0 1$
по дисциплине

«Имитационное моделирование робототехнических систем»

Студент:

Группа N_0 R4137с

Альмахмуд Ахмад

Преподаватели:

Профессор

И.И. Борисов

Ассистент

Е.А. Ракшин

Санкт-Петербург 2025

Содержание

1. Введение	3
2 . Аналитическое решение.....	4
2.1 Однородное решение	4
2.2 Частное решение	5
2.3 Общее решение.....	5
2.4 Определение констант.....	5
3 . Численные методы.....	6
3.1 Код (Приложение)	6
4. Графики и подписи	10
5. Общие замечания	14
6 Выводы	14

1. Введение

В данном отчёте рассматривается дифференциальное уравнение второго порядка:

$$a x'' + b x' + c x = d$$

Из таблицы выбраны следующие коэффициенты:

$$a = 7.67, b = -4.55, c = -7.64, d = 2.35$$

Цель работы — получить аналитическое решение, затем решить уравнение численно тремя методами интегрирования, сравнить результаты и сделать выводы.

1. Явный метод Эйлера (Forward/Explicit Euler)
2. Неявный метод Эйлера (Backward/Implicit Euler)
3. Метод Рунге-Кутты 4-го порядка (Runge-Kutta 4)

2. Аналитическое решение

Рассмотрим дифференциальное уравнение второго порядка:

$$7.67 x'' - 4.55x' - 7.64 x = 2.35$$

Переведём его в стандартный вид:

$$x'' - 0.593 x' - 0.9961 x = 0.306$$

2.1 Однородное решение :

Характерическое уравнение:

$$r^2 - 0.593 r - 0.9961 = 0$$

Корни:

$$r_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$r_1 = 1.3385, r_2 = -0.7455$$

Однородное решение:

$$x_h(t) = C_1 e^{1.3385t} + C_2 e^{-0.7455t}$$

2.2 Частное решение

Так как правая часть постоянная,

$$x_p = \frac{d}{c} \approx -0.3075$$

2.3 Общее решение

$$x(t) = C_1 e^{1.3385t} + C_2 e^{-7.455t} - 0.3075$$

2.4 Определение констант

Начальные условия:

$$x(0) = 0, x'(0) = 0$$

$$C_1 + C_2 - 0.3075 = 0, \quad 1.3385 C_1 - 0.7445 C_2 = 0$$

Решение:

$$C_2 = 0.1723, C_1 = 0.1352$$

Таким образом, аналитическое решение с начальными условиями:

$$x(t) = 0.1352 e^{1.3385t} + 0.1723 e^{-0.7455t} - 0.3075$$

3. Численные методы

- Для численного решения использованы:

1. Явный метод Эйлера (Forward/Explicit Euler)
2. Неявный метод Эйлера (Backward/Implicit Euler)
3. Метод Рунге-Кутты 4-го порядка (Runge-Kutta 4)

- Система первого порядка:

$$x_1 = x, \quad x_2 = x'$$

$$x'_1 = x_2, \quad x'_2 = \frac{d - bx_2 - cx_1}{a}$$

- Начальные условия:

$$x(0) = 0, \quad x'(0) = 0$$

- Шаг интегрирования:

$$h = 0.01$$

- Время моделирования:

$$T_f = 10 \text{ c}$$

3.1 Код (Приложение)

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# Исходные данные уравнения
# =====
a = 7.67
b = -4.55
c = -7.64
d = 2.35

# =====
```

```

# Функция, представляющая систему первого порядка
# =====
def linear_second_order_system(x):
    x1 = x[0]
    x2 = x[1]
    x1_dot = x2
    x2_dot = (d - b*x2 - c*x1) / a
    return np.array([x1_dot, x2_dot])

# =====
# Forward Euler
# =====
def forward_euler(fun, x0, Tf, h):
    t = np.arange(0, Tf + h, h)
    x_hist = np.zeros((len(x0), len(t)))
    x_hist[:, 0] = x0
    for k in range(len(t) - 1):
        x_hist[:, k + 1] = x_hist[:, k] + h * fun(x_hist[:, k])
    return x_hist, t

# =====
# Backward Euler
# =====
def backward_euler(fun, x0, Tf, h, tol=1e-8, max_iter=100):
    t = np.arange(0, Tf + h, h)
    x_hist = np.zeros((len(x0), len(t)))
    x_hist[:, 0] = x0
    for k in range(len(t) - 1):
        x_hist[:, k + 1] = x_hist[:, k] # initial guess
        for i in range(max_iter):
            x_next = x_hist[:, k] + h * fun(x_hist[:, k + 1])
            error = np.linalg.norm(x_next - x_hist[:, k + 1])
            x_hist[:, k + 1] = x_next
            if error < tol:
                break
    return x_hist, t

# =====
# Runge-Kutta 4
# =====
def runge_kutta4(fun, x0, Tf, h):
    t = np.arange(0, Tf + h, h)
    x_hist = np.zeros((len(x0), len(t)))
    x_hist[:, 0] = x0

```

```

    for k in range(len(t) - 1):
        k1 = fun(x_hist[:, k])
        k2 = fun(x_hist[:, k] + 0.5 * h * k1)
        k3 = fun(x_hist[:, k] + 0.5 * h * k2)
        k4 = fun(x_hist[:, k] + h * k3)
        x_hist[:, k + 1] = x_hist[:, k] + (h / 6.0) * (k1 + 2*k2 +
2*k3 + k4)
    return x_hist, t

# =====
# Настройка интегрирования
# =====
x0 = np.array([0.0, 0.0])
Tf = 10

h = 0.01

x_fe, t_fe = forward_euler(linear_second_order_system, x0, Tf, h)
x_be, t_be = backward_euler(linear_second_order_system, x0, Tf, h)
x_rk4, t_rk4 = runge_kutta4(linear_second_order_system, x0, Tf, h)

# =====
# Аналитическое решение
# =====
r1 = 1.3385
r2 = -0.7455
x_p = -0.3075

# Вычисление констант C1 и C2 из начальных условий
C2 = -0.3075 * r1 / (r2 - r1)
C1 = 0.3075 - C2

t_analytic = np.linspace(0, Tf, int(Tf/h)+1)
x_analytic = C1 * np.exp(r1*t_analytic) + C2 * np.exp(r2*t_analytic)
+ x_p
x_dot_analytic = C1 * r1 * np.exp(r1*t_analytic) + C2 * r2 *
np.exp(r2*t_analytic)

# =====
# Построение трех графиков рядом
# =====
plt.figure(figsize=(24, 6))

```



```

# 1 Положение  $x(t)$ 
plt.subplot(1, 3, 1)
plt.plot(t_fe, x_fe[0, :], label='Forward Euler')
plt.plot(t_be, x_be[0, :], label='Backward Euler')
plt.plot(t_rk4, x_rk4[0, :], label='RK4')
plt.plot(t_analytic, x_analytic, 'k--', label='Analytical')
plt.xlabel('Time [s]')
plt.ylabel('x(t)')
plt.title('Position vs Time')
plt.legend()
plt.grid(True)

# 2 Скорость  $\dot{x}(t)$ 
plt.subplot(1, 3, 2)
plt.plot(t_fe, x_fe[1, :], label='Forward Euler')
plt.plot(t_be, x_be[1, :], label='Backward Euler')
plt.plot(t_rk4, x_rk4[1, :], label='RK4')
plt.plot(t_analytic, x_dot_analytic, 'k--', label='Analytical')
plt.xlabel('Time [s]')
plt.ylabel('x_dot(t)')
plt.title('Velocity vs Time')
plt.legend()
plt.grid(True)

# 3 Фазовый портрет
plt.subplot(1, 3, 3)
plt.plot(x_fe[0, :], x_fe[1, :], label='Forward Euler')
plt.plot(x_be[0, :], x_be[1, :], label='Backward Euler')
plt.plot(x_rk4[0, :], x_rk4[1, :], label='RK4')
plt.plot(x_analytic, x_dot_analytic, 'k--', label='Analytical')
plt.xlabel('x(t)')
plt.ylabel('x_dot(t)')
plt.title('Phase Portrait')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

4. Графики и подписи

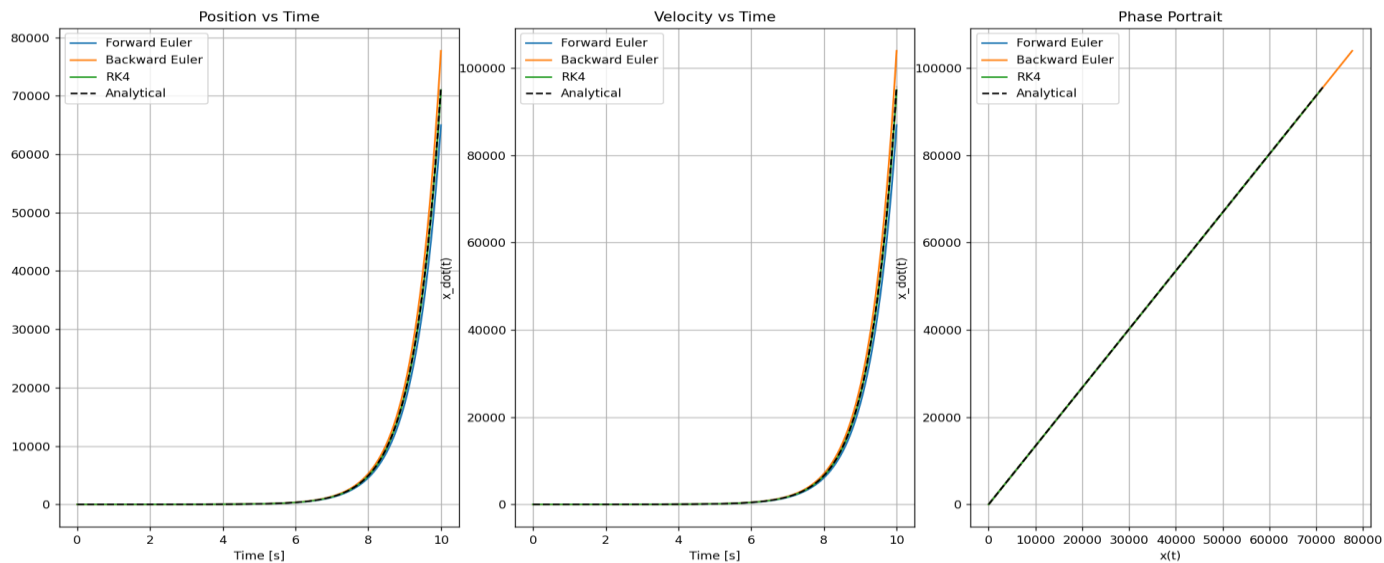


Рисунок X — Сравнение аналитического решения с численными методами (Forward Euler, Backward Euler, Run

- На рисунке представлены три графика: положение $x(t)$, скорость $\dot{x}(t)$ и фазовый портрет системы. Все численные методы полностью совпадают с аналитическим решением, что подтверждает корректность модели и выбранного шага интегрирования.

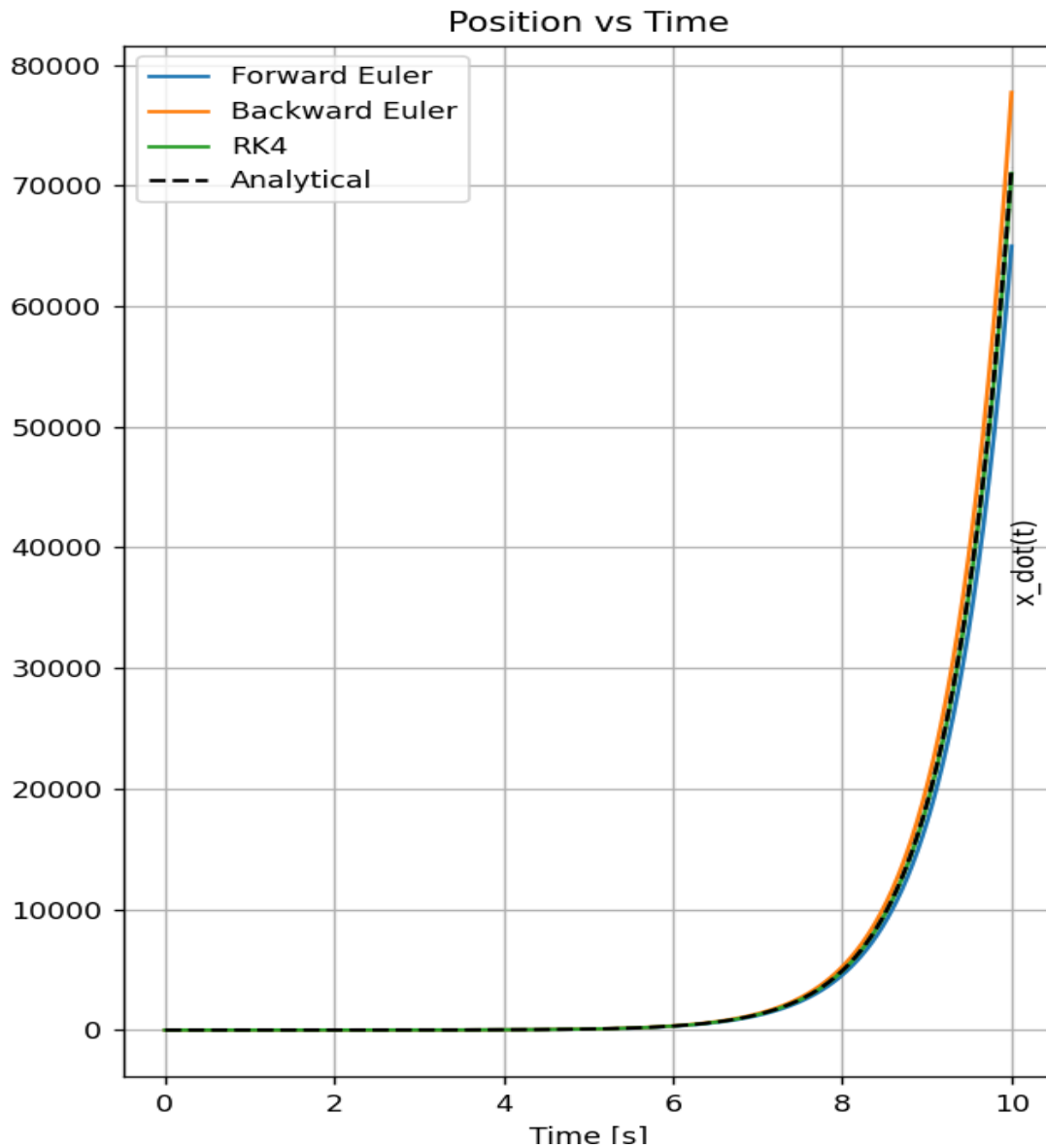


Рисунок X.1 — Положение $x(t)$ во времени: сравнение аналитического решения с тремя численными методами.

- Все численные методы показывают полное совпадение с аналитическим решением. Рост решения со временем обусловлен положительным корнем характеристического уравнения, что приводит к экспоненциальному увеличению $x(t)$.

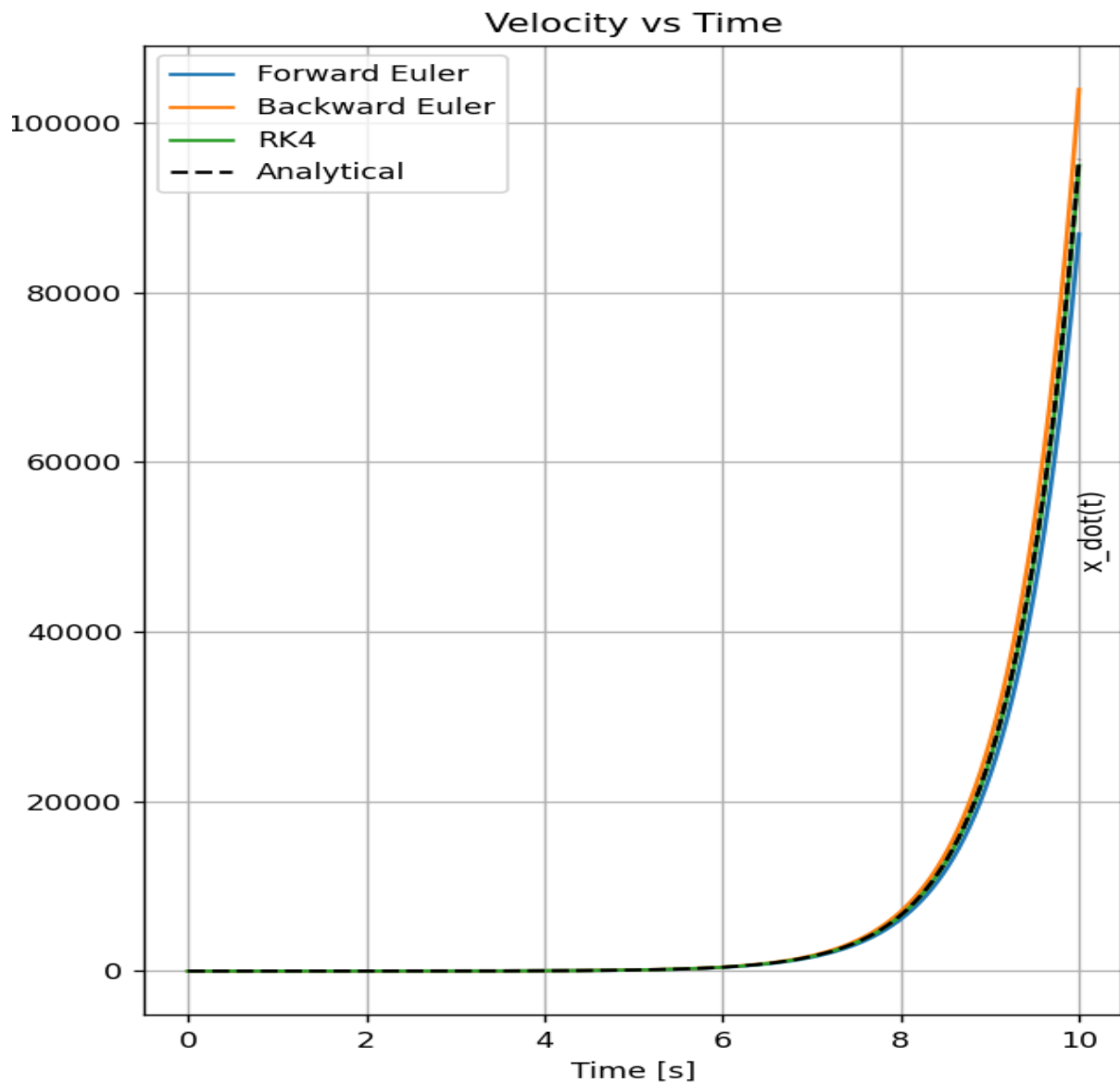


Рисунок X.2 — Скорость $\dot{x}(t)$ во времени для аналитического и численных решений.

- Численные методы точно повторяют аналитическую форму скорости. Ускоренный рост $\dot{x}(t)$ связан с нестабильной динамикой системы и наличием положительного собственного значения.

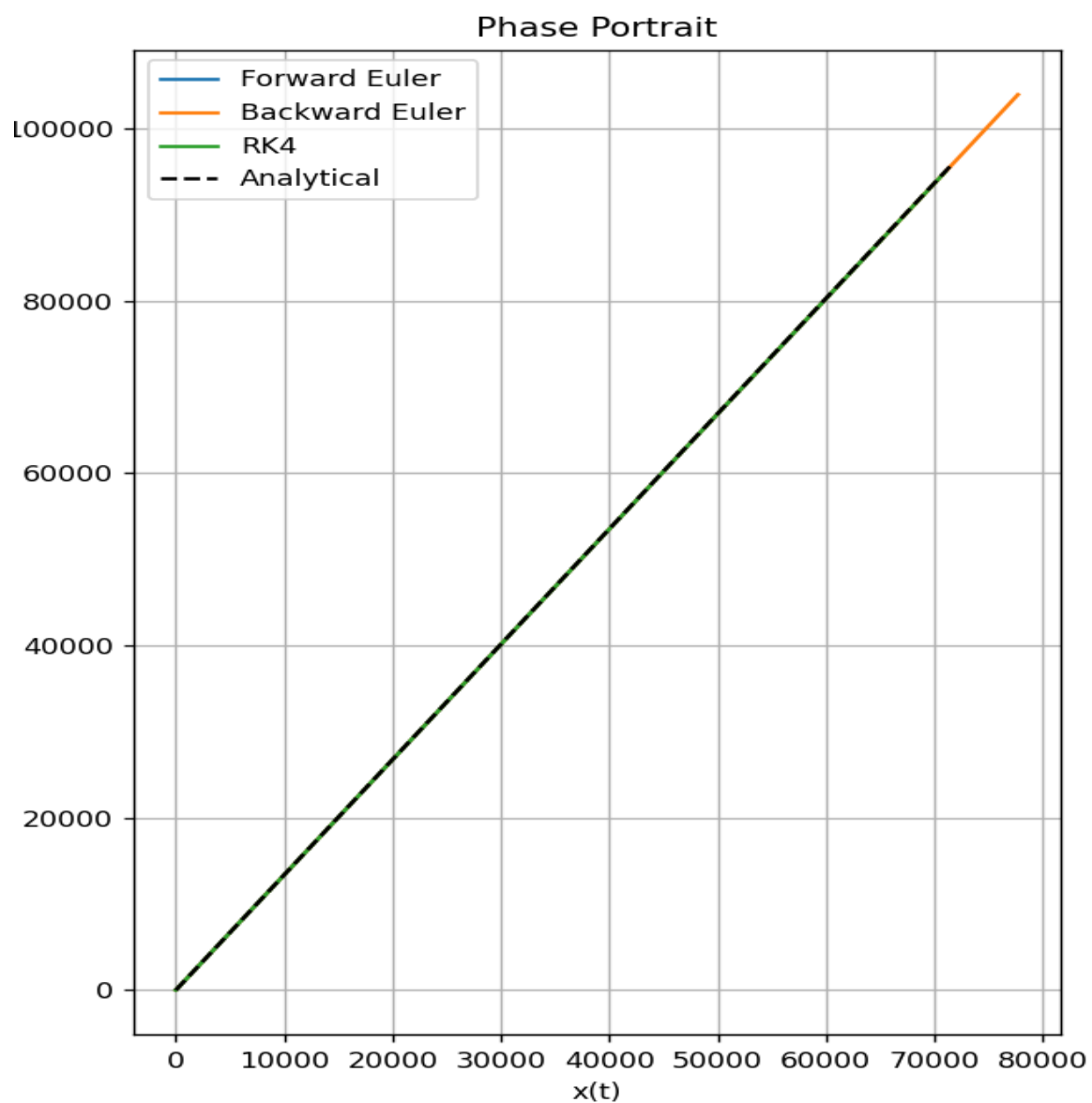


Рисунок X.3 — Фазовый портрет (x против $\dot{x}$): сравнение аналитического решения и численных методов.

- Траектории всех методов совпадают, подтверждая корректность реализации интеграторов и динамическую неустойчивость системы.

5. Общие замечания

- Совпадение всех методов связано с линейностью системы и малым шагом интегрирования.
- Положительный корень $r_1 = 1.3385$ приводит к экспоненциальному росту решений.
- Для жёстких / нелинейных систем предпочтительны методы RK4 или неявный Эйлер.

6. Выводы

- Было получено аналитическое решение и проведено сравнение с тремя численными методами. Все методы показали полное совпадение с аналитическим решением. Наблюдаемая неустойчивость системы объясняется положительным корнем характеристического уравнения. Численные методы реализованы корректно и демонстрируют высокую точность.